

**INSTITUTO
FEDERAL**
Santa Catarina

Introdução ao React

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

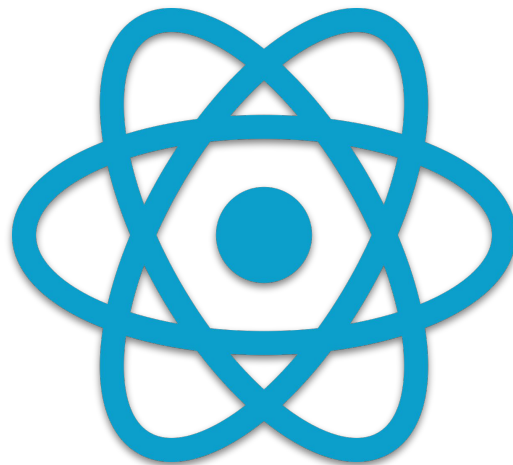
Roboto Mono



React

sobre

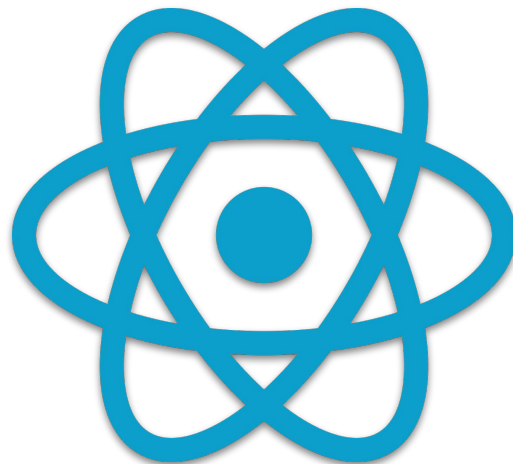
- biblioteca JavaScript de código aberto para criar interfaces de usuário (UI)
- baseado no conceito de componentes reutilizáveis
- utiliza um "DOM virtual" para tornar a atualização da interface mais eficiente
- criado para lidar com interfaces dinâmicas complexas e gerenciamento do estado da aplicação



React

histórico

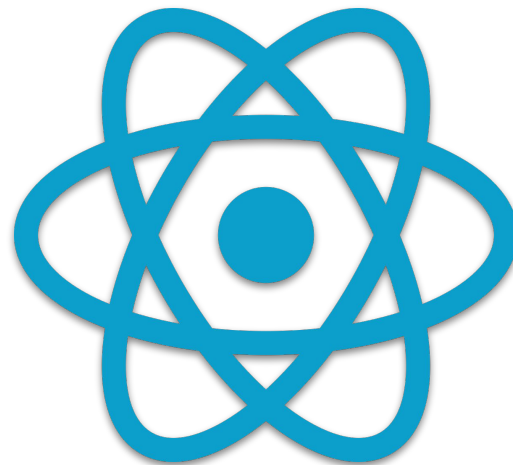
- **2011** desenvolvido por Jordan Walke (Facebook) inspirado por conceitos de programação funcional e a biblioteca XHP
 - **primeira aplicação** *feed* de notícias do Facebook
- **2013** apresentado ao público como open source
- **2014** começou a ser usado pelo Instagram
- **2015** lançamento do React Native
- **2016-2017** ganha popularidade com a adoção pelo Netflix, Airbnb e Uber



React

características

- declarativa
- componentização
- unidirecionalidade dos dados
- base em JavaScript



Criando um Novo Projeto

pré-requisito node.js com npm

create-react-app (CRA)

1. instalar via npx

- `npx create-react-app <nome-projeto>`

2. entrar no diretório do projeto

- `cd <nome-projeto>`

3. executar projeto

- `npm start`

vite

1. instalar via npm, informando os dados

- `npm create vite@latest`
 - nome
 - framework/biblioteca
 - variante

2. entrar no diretório do projeto e instalar dependências

- `cd <nome-projeto>`
- `npm install`

3. executar projeto

- `npm run dev`



JSX

o que é

- JavaScript XML
- extensão de sintaxe do JavaScript
- "mistura" JS e HTML
- ajuda a escrever a estrutura da interface de usuário de maneira semelhante ao HTML
- paradigma declarativo
- JSX com TS => TSX

exemplos

```
// JSX
const Saudacao = () => {
  return <h1>Olá, mundo!</h1>;
};

// JS
const Saudacao = () => {
  return React.createElement("h1", null, "Olá, mundo!");
};
```



JSX

características

- atributos em camelCase
- className para o atributo class
- expressões JS dentro de chaves {}

exemplos

```
<button onClick={handleClick}>Clique aqui</button>
```

```
<p className="destaque">Texto em destaque</p>
```

```
const nome = "Carlos";  
const Saudacao = () => {  
  return <h1>Olá, {nome}</h1>;  
};
```



Componentes

o que são

- base da construção das interfaces
- reutilizáveis, independentes e encapsulados
- podem ser simples ou complexos

tipos

- componentes de classe
- componentes funcionais

exemplos

```
// classe
import React, { Component } from "react";

class Saudacao extends Component {
  render() {
    return <h1>Olá, {this.props.nome}</h1>;
  }
}

export default Saudacao;

// função
import React from "react";

const Saudacao = ({ nome }) => {
  return <h1>Olá, {nome}</h1>;
};

export default Saudacao;
```



Componentes

props

- valores passados de um componente pai para um componente filho
- imutáveis dentro do componente recebe
- pode ser desestruturado

exemplos

```
// Componente pai
const Pai = () => {
  const mensagem = "Olá!";
  return <Filho texto={mensagem} />;
};
```

```
// Componente filho
const Filho = (props) => {
  return <p>{props.texto}</p>;
};
```

```
// Componente filho com prop desestruturada
const Filho = ({ texto }) => {
  return <p>{texto}</p>;
};
```



Componentes

props

- valores passados de um componente pai para um componente filho
- imutáveis dentro do componente recebe
- pode ser desestruturado

exemplos

```
// Componente pai
<Usuario nome="Ana" idade={25} cidade="São José"/>
```

```
// Componente filho
const Usuario = (props) => {
  return (
    <p>{props.nome} tem {props.idade} anos e
      mora em {props.cidade}</p>
  );
};
```

```
// Componente filho com prop desestruturada
const Usuario = ({ nome, idade, cidade }) => {
  return (
    <p>{nome} tem {idade} anos e mora em {cidade}</p>
  );
};
```



Componentes

props

- valores passados de um componente pai para um componente filho
- imutáveis dentro do componente recebe
- pode ser desestruturado
- pode ter um valor padrão

exemplos

```
// Componente pai
<Usuario /> // Exibe "Bem-vindo, Visitante!"

// Componente filho
const Usuario = ({ nome = "Visitante" }) => {
  return <p>Bem-vindo, {nome}!</p>;
};
```



Componentes

estados

- objetos que contêm dados que podem mudar
- afetam o que é exibido na tela
- quando o estado de um componente muda, ele é re-renderizado para refletir a mudança

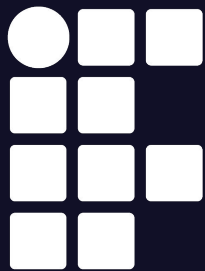
exemplos

```
import { useState } from "react";

const Contador = () => {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Valor: {contador}</p>
      <button onClick={() =>
        setContador(contador + 1)}>Incrementar</button>
    </div>
  );
};
```





**INSTITUTO
FEDERAL**
Santa Catarina

Introdução ao React

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

